

Module 4: Node.js, Express & TypeORM > Section 12: Query Builder, Migrations & Transactions > 12-01: Query Builder

Query Builder in TypeORM

Section	12 — Query Builder, Migrations, and Transactions in TypeORM
Subtopic	12-01 — Query Builder
Duration	~2–3 hours (within the 5–7 hr section allocation)
Tags	Theory Practical TypeScript
Topics covered	QueryBuilder entry points, SELECT with WHERE / ORDER BY / LIMIT, JOINS, INSERT / UPDATE / DELETE builders, raw SQL integration, subqueries
Prerequisites	11-01 (Entities), 11-02 (Relations), 11-03 (Repository CRUD & FindOptions)

Learning Objectives — This Subtopic

- Explain when QueryBuilder is the appropriate tool compared to the Repository `find` API.
- Construct SELECT queries using `createQueryBuilder` with `where`, `orderBy`, `limit`, `offset`, and parameter binding.
- Join related entities using `innerJoin`, `leftJoin`, `innerJoinAndSelect`, and `leftJoinAndSelect`.
- Use QueryBuilder for INSERT, UPDATE, and DELETE operations.
- Embed raw SQL expressions safely using `Brackets` and typed parameters.
- Compose correlated subqueries using a nested QueryBuilder instance.

1. When the Repository API Is Not Enough

The repository `find` API (Section 11-03) covers the majority of everyday queries. It stops being sufficient when you need any of the following:

- Aggregate functions — `COUNT`, `SUM`, `AVG`, `MAX` — with or without `GROUP BY` and `HAVING`.
- Joining across entities where the join condition is not a simple relation (e.g., joining on a computed expression).
- Selecting individual columns from multiple entities into a flat result shape.
- Constructing a query dynamically — different `WHERE` clauses depending on which filters the user supplies.
- Correlated subqueries.

QueryBuilder fills that gap. It is a fluent API that composes an SQL statement programmatically and hands it off to the database driver, returning typed results.

Analogy: The repository `find` API is like ordering from a fixed menu — fast, clear, covers 80% of meals. QueryBuilder is the open kitchen where you can ask for any combination of ingredients. You gain full control at the cost of writing more code.

Key Insight: QueryBuilder and the repository API are not mutually exclusive. A custom repository method (Section 11-03) can delegate to QueryBuilder internally while exposing a clean, named method to the rest of the application. This is the recommended pattern: hide QueryBuilder complexity behind a repository method boundary.

2. Reference Entities

All examples in this document use the two entities below. They represent a simple blog domain: a `User` who authors many `Post` records.

```
src/entity/User.ts
```

```
import { Entity, PrimaryGeneratedColumn, Column, CreateDateColumn, OneToMany } from "typeorm";
import { Post } from "../Post";

@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({ length: 100 })
  name: string;

  @Column({ unique: true })
  email: string;

  @Column({ default: true })
  isActive: boolean;

  @CreateDateColumn()
  createdAt: Date;

  @OneToMany(() => Post, (post) => post.author)
  posts: Post[];
}
```

src/entity/Post.ts

```
import { Entity, PrimaryGeneratedColumn, Column, ManyToOne, CreateDateColumn } from "typeorm";
import { User } from "../User";

@Entity()
export class Post {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  title: string;

  @Column("text")
  body: string;

  @Column({ default: false })
  published: boolean;

  @Column({ default: 0 })
  viewCount: number;
}
```

```
@CreateDateColumn()
createdAt: Date;

@ManyToOne(() => User, (user) => user.posts, { nullable: false })
author: User;
}
```

These entities are not runnable on their own. The next section introduces QueryBuilder with these as the basis for all examples.

3. Creating a QueryBuilder — Entry Points

There are three ways to obtain a QueryBuilder instance. All produce equivalent results; the choice is a matter of context.

3.1 From a Repository

The most common pattern in service classes — you already have the repository and simply call `.createQueryBuilder(alias)` on it.

src/query-demo.ts

```
import { AppDataSource } from "../data-source";
import { Post } from "../entity/Post";

const postRepo = AppDataSource.getRepository(Post);

// "post" is the alias – the SQL table alias used in all subsequent clauses
const qb = postRepo.createQueryBuilder("post");
```

3.2 From the DataSource

When you do not have a repository to hand, or you need to start a builder for an entity you did not inject:

src/query-demo.ts

```
const qb = AppDataSource
  .createQueryBuilder()
  .select("post")
  .from(Post, "post");
```

3.3 From the Entity Manager

```
src/query-demo.ts
```

```
const qb = AppDataSource.manager.createQueryBuilder(Post, "post");
```

Key Insight: The alias ("post" in the examples above) is the SQL table alias. Every subsequent clause — `where`, `orderBy`, `leftJoin` — references columns through this alias. Choosing a short, meaningful alias avoids confusion when joining multiple tables.

4. SELECT Queries

4.1 Fetching All Rows

STAGE 1

The simplest QueryBuilder query — select every post and return typed `Post` entities.

```
src/query-demo.ts
```

```
const posts = await AppDataSource
  .getRepository(Post)
  .createQueryBuilder("post")
  .getMany();

console.log("Total posts:", posts.length);
```

Output:

Total posts: 5

`getMany()` executes the query and returns `Post[]`. The companion method `getOne()` returns `Post | null`. If you want the raw SQL sent to the database, call `getSql()` instead — it returns the query string without executing it, which is invaluable during development.

4.2 WHERE — Filtering with Parameters

STAGE 2

Important — Never interpolate user input directly into a WHERE string. String interpolation bypasses the driver's parameterisation and introduces SQL injection. Always use named parameters (`:paramName`) and pass values through the second argument of `where` / `andWhere` .

src/query-demo.ts

```
const published = await AppDataSource
  .getRepository(Post)
  .createQueryBuilder("post")
  .where("post.published = :pub", { pub: true })
  .andWhere("post.viewCount > :minViews", { minViews: 100 })
  .getMany();

console.log("Published + popular:", published.length);
```

Output:

Published + popular: 2

What is happening? Each `andWhere` call appends an `AND` condition. Use `orWhere` for an `OR` branch. The named parameters (`:pub` , `:minViews`) are sent as prepared statement parameters — the driver substitutes them safely.

4.3 ORDER BY, LIMIT, and OFFSET

STAGE 3

src/query-demo.ts

```
const page = 2;
const pageSize = 3;

const paginated = await AppDataSource
  .getRepository(Post)
  .createQueryBuilder("post")
  .where("post.published = :pub", { pub: true })
  .orderBy("post.createdAt", "DESC")
  .addOrderBy("post.title", "ASC")          // secondary sort
  .limit(pageSize)
  .offset((page - 1) * pageSize)
  .getMany();

console.log("Page 2 posts:", paginated.map(p => p.title));
```

Output:

```
Page 2 posts: [ 'Async Patterns', 'Node Streams' ]
```

4.4 Selecting Specific Columns

By default QueryBuilder selects all columns. Use `select` to restrict which columns come back. This is useful for list endpoints where you do not need large text columns.

src/query-demo.ts

```
const titles = await AppDataSource
  .getRepository(Post)
  .createQueryBuilder("post")
  .select(["post.id", "post.title", "post.viewCount"])
  .orderBy("post.viewCount", "DESC")
  .getMany();

titles.forEach(p => console.log(`[${p.id}] ${p.title} - ${p.viewCount} views`));
```

Output:

[3] TypeORM Relations — 450 views
 [1] Node.js Event Loop — 312 views
 [5] Express Middleware — 200 views
 [2] Async Patterns — 98 views
 [4] Node Streams — 34 views

Key Insight: When you restrict columns with `select`, the unselected properties on the returned entity instances are `undefined` — they are not populated. This is expected behaviour, not an error. Document this clearly in any service method that returns partial entities, or use a DTO type rather than the entity class to signal that the object is incomplete.

5. Joining Related Entities

QueryBuilder provides four join methods. The distinction between them is whether the joined entity is included in the result object (*AndSelect* variants) or used purely as a filter/condition (*bare* variants).

Method	SQL type	Populates result?
<code>innerJoin</code>	INNER JOIN	No — only filters
<code>leftJoin</code>	LEFT JOIN	No — only filters
<code>innerJoinAndSelect</code>	INNER JOIN	Yes — nested in entity
<code>leftJoinAndSelect</code>	LEFT JOIN	Yes — nested in entity

5.1 leftJoinAndSelect — Loading a Relation

STAGE 1

Return posts and embed the full author `user` object on each post. Equivalent to the `relations: { author: true }` option in `findOne`, but composable with the rest of the QueryBuilder chain.

```
src/query-demo.ts
```



```
const withAuthors = await AppDataSource
  .getRepository(Post)
  .createQueryBuilder("post")
  .leftJoinAndSelect("post.author", "author")
  .where("post.published = :pub", { pub: true })
  .getMany();

withAuthors.forEach(p =>
  console.log(`${p.title} by ${p.author.name}`)
);
```

Output:

```
"TypeORM Relations" by Carol Smith
"Express Middleware" by Carol Smith
"Node.js Event Loop" by Dave
```

What is happening? The first argument to `leftJoinAndSelect` is the *relation path* — the alias of the root entity followed by the property name declared on the entity. The second argument is the alias for the joined table, used in subsequent clauses.

5.2 innerJoin (without select) — Filter via Relation

STAGE 2

Find all posts written by users who are currently active. The `user` data itself is not needed in the result — only the posts.

src/query-demo.ts

```
const postsFromActiveAuthors = await AppDataSource
  .getRepository(Post)
  .createQueryBuilder("post")
  .innerJoin("post.author", "author") // INNER JOIN – excludes posts with no
  author
  .where("author.isActive = :active", { active: true })
  .orderBy("post.createdAt", "DESC")
  .getMany();

console.log("Posts from active authors:", postsFromActiveAuthors.length);
```

Output:

Posts from active authors: 4

Because only `innerJoin` (not `innerJoinAndSelect`) is used, `post.author` is **not** populated on the returned entities. The join exists purely to drive the WHERE condition against the `author` alias.

5.3 Multi-Level Joins

Joins are chainable. Each join adds an alias that subsequent joins and WHERE clauses can reference.

src/query-demo.ts

```
// Hypothetical: Post → Comment → Author (three levels)
// Each join uses the alias from the previous join as its left-hand side.
const deep = await AppDataSource
  .getRepository(Post)
  .createQueryBuilder("post")
  .leftJoinAndSelect("post.author", "author")
  .leftJoinAndSelect("post.comments", "comment") // requires Post.comments relation
  .leftJoinAndSelect("comment.author", "commentAuthor")
  .where("post.published = :pub", { pub: true })
  .getMany();

console.log("Posts loaded:", deep.length);
```

Output:

Posts loaded: 3

Important: Joining three or four levels deep in a single query can produce a very large result set due to the Cartesian product of joined rows. For deeply nested data, consider loading the root entity first and then fetching nested relations separately, or use the QueryBuilder's `take` and `skip` on the root entity only (pagination on a query with JOINS requires care — see §8).

6. Aggregates, GROUP BY, and HAVING

Aggregate functions are one of the primary reasons to reach for QueryBuilder instead of the repository API. Use `addSelect` to include aggregate expressions, and `groupBy` / `addGroupBy` to partition rows.

6.1 Counting Posts per Author

src/query-demo.ts

```
const authorStats = await AppDataSource
  .getRepository(User)
  .createQueryBuilder("user")
  .leftJoin("user.posts", "post")
  .select("user.id", "userId")
  .addSelect("user.name", "userName")
  .addSelect("COUNT(post.id)", "postCount")
  .groupBy("user.id")
  .addGroupBy("user.name")
  .orderBy("postCount", "DESC")
  .getRawMany<{ userId: number; userName: string; postCount: string }>();

authorStats.forEach(row =>
  console.log(`${row.userName}: ${row.postCount} posts`)
);
```

Output:

```
Carol Smith: 3 posts
Dave: 2 posts
Eve: 0 posts
```

What is happening? When you project raw expressions (like `COUNT(...)`), the result no longer maps cleanly to a typed entity — you get a flat object instead. `getRawMany` returns `any[]` by default, but you can pass a generic type argument to tell TypeScript what shape to expect. Note that SQL aggregate results are always returned as strings by most drivers (even for numeric functions); cast with `Number(...)` when needed.

6.2 HAVING — Filtering on Aggregates

src/query-demo.ts

```
// Authors with more than 1 published post
const prolific = await AppDataSource
  .getRepository(User)
  .createQueryBuilder("user")
  .innerJoin("user.posts", "post")
  .select("user.name", "userName")
  .addSelect("COUNT(post.id)", "postCount")
  .where("post.published = :pub", { pub: true })
  .groupBy("user.id")
  .addGroupBy("user.name")
  .having("COUNT(post.id) > :min", { min: 1 })
  .getRawMany<{ userName: string; postCount: string }>();

prolific.forEach(r => console.log(r.userName, "→", r.postCount, "posts"));
```

Output:

Carol Smith → 2 posts

7. Dynamic WHERE Clauses and Brackets

A common real-world pattern is building a query where some filters are optional — applied only when the user provides them. QueryBuilder is well suited to this because you can conditionally call `andWhere`.

7.1 Conditionally Applied Filters

src/query-demo.ts

```
interface PostFilters {
  published?: boolean;
  authorId?: number;
  search?: string;
}

async function searchPosts(filters: PostFilters) {
  const qb = AppDataSource
    .getRepository(Post)
    .createQueryBuilder("post")
```

```
.leftJoinAndSelect("post.author", "author");

if (filters.published !== undefined) {
  qb.andWhere("post.published = :pub", { pub: filters.published });
}

if (filters.authorId !== undefined) {
  qb.andWhere("author.id = :authorId", { authorId: filters.authorId });
}

if (filters.search) {
  qb.andWhere("LOWER(post.title) LIKE :search", {
    search: `%${filters.search.toLowerCase()}%`,
  });
}

return qb.orderBy("post.createdAt", "DESC").getMany();
}

// Usage
const results = await searchPosts({ published: true, search: "node" });
results.forEach(p => console.log(p.title));
```

Output:

Node.js Event Loop
Node Streams

7.2 Grouping Conditions with Brackets

When you need to express `(A OR B) AND C`, use TypeORM's `Brackets` class to wrap the OR group.

src/query-demo.ts

```
import { Brackets } from "typeorm";

const results = await AppDataSource
  .getRepository(Post)
  .createQueryBuilder("post")
  .where("post.published = :pub", { pub: true })
  .andWhere(new Brackets(qb => {
    qb.where("post.viewCount > :high", { high: 300 })
      .orWhere("post.viewCount = :zero", { zero: 0 });
  }));
```

```
    )))  
    .getMany();  
  
results.forEach(p => console.log(p.title, p.viewCount));
```

Output:

```
TypeORM Relations 450  
Node.js Event Loop 312
```

What is happening? The SQL generated is: `WHERE post.published = true AND (post.viewCount > 300 OR post.viewCount = 0)`. Without Brackets, chaining `orWhere` would produce `WHERE ... AND ... OR ...`, which has different precedence semantics and is almost never what you intend.

Important: Never chain `orWhere` at the top level when you have other `andWhere` conditions. SQL's operator precedence means the `OR` will bind loosely and likely include rows you did not intend. Always wrap OR groups in Brackets.

8. INSERT, UPDATE, and DELETE with QueryBuilder

QueryBuilder is not limited to SELECT. The same fluent API covers the full DML surface, which is useful when you need to perform bulk operations or use database-level expressions in the SET clause.

8.1 INSERT

src/query-demo.ts

```
import { InsertResult } from "typeorm";  
  
const result: InsertResult = await AppDataSource  
    .createQueryBuilder()  
    .insert()  
    .into(Post)  
    .values([  
        { title: "QueryBuilder Deep Dive", body: "...", published: false, author: { id: 1 } },
```

```
{ title: "TypeORM Migrations",    body: "...", published: false, author: { id: 1 } },
])
.execute();

console.log("Inserted ids:", result.identifiers);
```

Output:

```
Inserted ids: [ { id: 6 }, { id: 7 } ]
```

`execute()` runs the query and returns an `InsertResult` containing the generated identifiers. It does not return entity instances — use `getRepository(Post).findBy({ id: In([6, 7]) })` if you need the full entities back.

8.2 UPDATE

QueryBuilder UPDATE is the most efficient way to apply a database-level expression to many rows at once (e.g., incrementing a counter).

src/query-demo.ts

```
import { UpdateResult } from "typeorm";

// Increment viewCount by 1 for a specific post – no round-trip fetch required
const update: UpdateResult = await AppDataSource
  .createQueryBuilder()
  .update(Post)
  .set({ viewCount: () => "viewCount + 1" }) // arrow fn = raw SQL expression
  .where("id = :id", { id: 3 })
  .execute();

console.log("Rows updated:", update.affected);
```

Output:

```
Rows updated: 1
```

What is happening? Passing an arrow function as the value in `set` tells TypeORM to embed the string directly as a SQL expression rather than a parameter. This is how you issue

```
SET viewCount = viewCount + 1
```

 without reading the row first.

8.3 DELETE

src/query-demo.ts

```
import { DeleteResult } from "typeorm";

// Hard-delete all unpublished posts older than 30 days
const thirtyDaysAgo = new Date();
thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);

const del: DeleteResult = await AppDataSource
  .createQueryBuilder()
  .delete()
  .from(Post)
  .where("published = :pub", { pub: false })
  .andWhere("createdAt < :cutoff", { cutoff: thirtyDaysAgo })
  .execute();

console.log("Deleted:", del.affected, "posts");
```

Output:

Deleted: 1 posts

9. Raw SQL Integration

Occasionally a database-specific function or expression has no TypeORM equivalent. TypeORM provides two escape hatches.

9.1 Inline Expressions in WHERE

You can write any valid SQL fragment inside a `where` or `andWhere` string, as long as column references use the alias prefix and values go through named parameters.

src/query-demo.ts

```
// Use a database function (SQLite's date() – adjust for your target DB)
const recent = await AppDataSource
  .getRepository(Post)
  .createQueryBuilder("post")
  .where("date(post.createdAt) = date('now')")
  .getMany();

console.log("Posted today:", recent.length);
```

Output:

Posted today: 2

9.2 query — Full Raw SQL

For queries that QueryBuilder cannot express at all, `dataSource.query(sql, params)` sends raw SQL and returns plain row objects. Use this sparingly — it bypasses TypeScript typing and all TypeORM safeguards.

src/query-demo.ts

```
// Raw SQL with positional parameters (driver-specific: ? for SQLite, $1 for PostgreSQL)
const rows = await AppDataSource.query(
  `SELECT p.id, p.title, u.name AS authorName
   FROM post p
   INNER JOIN user u ON u.id = p.authorId
   WHERE p.viewCount > ?`,
  [200]
);

console.log(rows);
```

Output:

```
[
  { id: 1, title: 'Node.js Event Loop', authorName: 'Dave' },
  { id: 3, title: 'TypeORM Relations', authorName: 'Carol Smith' },
  { id: 5, title: 'Express Middleware', authorName: 'Carol Smith' }
]
```

Warning: Raw SQL via `dataSource.query` is database-driver-specific. Positional parameter syntax differs — SQLite uses `?`, PostgreSQL uses `$1, $2`, MySQL uses `?` (in the same order as the array). Code written with raw SQL will need changes when switching databases. Prefer `QueryBuilder` wherever possible.

10. Subqueries

A subquery is a `QueryBuilder` instance embedded inside another `QueryBuilder`. `TypeORM` supports subqueries in `WHERE` and `FROM` positions.

10.1 Subquery in `WHERE` — `IN`

Find all users who have at least one published post, using a subquery to produce the list of qualifying author IDs.

STAGE 1

src/query-demo.ts

```
const subQuery = AppDataSource
  .getRepository(Post)
  .createQueryBuilder("post")
  .select("post.authorId")           // the column the outer query will match against
  .where("post.published = :pub", { pub: true });

const activeAuthors = await AppDataSource
  .getRepository(User)
  .createQueryBuilder("user")
  .where(`user.id IN (${subQuery.getQuery()})`)
  .setParameters(subQuery.getParameters()) // transfer parameters from sub to outer
  .getMany();

console.log("Authors with published posts:", activeAuthors.map(u => u.name));
```

Output:

Authors with published posts: ['Carol Smith', 'Dave']

What is happening? `subQuery.getQuery()` returns the SQL string (without executing it).

`subQuery.getParameters()` extracts the named parameter map. You must transfer the parameters to the outer query with `setParameters`, otherwise the driver receives a query referencing `:pub` with no binding.

10.2 Subquery in FROM — Derived Table

STAGE 2

For more advanced cases, a `QueryBuilder` can be used as a virtual table in the FROM clause.

src/query-demo.ts

```
// Treat aggregate results as an inline view and further filter on them
const subQuery = AppDataSource
  .getRepository(Post)
  .createQueryBuilder("post")
  .select("post.authorId", "authorId")
  .addSelect("SUM(post.viewCount)", "totalViews")
  .where("post.published = :pub", { pub: true })
  .groupBy("post.authorId");

const topAuthors = await AppDataSource
  .createQueryBuilder()
  .select("sub.authorId", "id")
  .addSelect("sub.totalViews", "views")
  .from(`(${subQuery.getQuery()})`, "sub")
  .setParameters(subQuery.getParameters())
  .where("CAST(sub.totalViews AS INTEGER) > :minViews", { minViews: 300 })
  .getRawMany<{ id: string; views: string }>();

topAuthors.forEach(r => console.log(`Author ${r.id}: ${r.views} total views`));
```

Output:

```
Author 1: 762 total views
Author 2: 312 total views
```

Key Insight: When using FROM subqueries, ensure you alias every selected expression in the subquery (`.select("...", "alias")`) because the outer query references columns by those aliases. Missing aliases produce ambiguous column name errors at runtime.

11. Inspecting and Debugging Generated SQL

Two methods help you understand what SQL TypeORM is generating before committing to running it.

src/query-demo.ts

```
const qb = AppDataSource
  .getRepository(Post)
  .createQueryBuilder("post")
  .where("post.published = :pub", { pub: true })
  .orderBy("post.createdAt", "DESC");

// Returns the SQL string with placeholders – does NOT execute
console.log("SQL:", qb.getSql());

// Returns { sql, parameters } – useful for logging
console.log("With params:", qb.getQueryAndParameters());
```

Output:

```
SQL: SELECT "post"."id" AS "post_id", "post"."title" AS "post_title", ... FROM "post" "post" WHERE
post.published = ? ORDER BY "post"."createdAt" DESC
With params: [
  'SELECT "post"."id" AS ... WHERE post.published = ? ORDER BY ...',
  [ true ]
]
```

You can also enable per-query logging by calling `.printSql()` on the builder — this pipes the generated SQL to the console at execution time without modifying the DataSource logging configuration.

src/query-demo.ts

```
const posts = await AppDataSource
  .getRepository(Post)
  .createQueryBuilder("post")
  .printSql() // logs SQL to console when getMany() fires
  .getMany();
```

Output:

```
[query] SELECT "post"."id" AS "post_id", "post"."title" AS "post_title", ... FROM "post" "post"
```

Summary Table

Concept	Key Point
Entry points	Obtain a builder via <code>repo.createQueryBuilder(alias)</code> , <code>dataSource.createQueryBuilder()</code> , or <code>manager.createQueryBuilder(Entity, alias)</code> . All are equivalent.
Alias	A short SQL table alias assigned at builder creation. All column references in subsequent clauses must be prefixed with the correct alias.
getMany / getOne	Execute the SELECT and return typed entity instances. <code>getRawMany</code> / <code>getRawOne</code> return plain objects for aggregate or projected queries.
Named parameters	Always use <code>:paramName</code> syntax and pass values as an object. Never interpolate user input into the query string.
leftJoinAndSelect vs leftJoin	

Concept	Key Point
	<i>AndSelect</i> variants populate the relation on the entity. Bare join variants filter without hydrating the joined data.
Brackets	Wraps an OR group so it binds before surrounding AND conditions. Essential for compound filters.
Aggregates	Use <code>addSelect("COUNT(...)", "alias")</code> , <code>groupBy</code> , and <code>having</code> . Results come via <code>getRawMany</code> with string-typed numeric fields.
UPDATE set with arrow function	<code>.set({ col: () => "col + 1" })</code> embeds a raw SQL expression — the only way to use database-level increment/decrement in QueryBuilder UPDATE.
Subqueries	Build with <code>getQuery()</code> + <code>getParameters()</code> ; embed in the outer query's WHERE string or FROM clause. Always transfer parameters with <code>setParameters</code> .
getSql / printSql	Development tools: <code>getSql()</code> returns the SQL string without executing; <code>printSql()</code> logs to console at execution time.

Interview Questions

1. When should you use QueryBuilder instead of the repository `find` API?

Expected answer: Use QueryBuilder when you need aggregate functions (COUNT, SUM, AVG), GROUP BY / HAVING, dynamic WHERE clauses built conditionally, projections across multiple joined entities, subqueries, or any SQL construct that the FindOptions API cannot express. The repository API is sufficient for straightforward equality / operator-based filtering, ordering, and relation loading.

2. What is the difference between `leftJoinAndSelect` and `leftJoin` ?

Expected answer: Both issue a SQL LEFT JOIN. `leftJoinAndSelect` additionally hydrates the joined entity and nests it on the parent entity instance (i.e., `post.author` is populated).

`leftJoin` performs the join purely for filtering — the joined data does not appear in the returned entities. Use the bare variant when you only need the join condition for a WHERE clause and do not need the related data in the result.

3. Why must you never interpolate user input directly into a QueryBuilder WHERE string?

Expected answer: String interpolation bypasses the database driver's prepared statement parameterisation, making the query vulnerable to SQL injection. TypeORM's named parameters (`:paramName` with values passed as a separate object) are sent to the driver as bound parameters and are never interpreted as SQL.

4. How do you express an OR condition inside a larger AND query without breaking operator precedence?

Expected answer: Import `Brackets` from `typeorm` and wrap the OR conditions inside a `Brackets` callback passed to `andWhere`. This generates `AND (condition1 OR condition2)` in SQL. Chaining `orWhere` at the top level generates `... AND ... OR ...`, which has unintended precedence — the OR binds loosely and can include rows from unrelated conditions.

5. How do you increment a numeric column by 1 using QueryBuilder UPDATE without first reading the row?

Expected answer: Pass an arrow function as the value in the `set` object: `.set({ viewCount: () => "viewCount + 1" })`. TypeORM detects the arrow function and embeds the string as a raw SQL expression rather than a bound parameter, producing `SET viewCount = viewCount + 1`.

6. What does `getRawMany` return versus `getMany` ?

Expected answer: `getMany` returns typed entity instances populated from the query result — TypeORM maps columns back to entity properties. `getRawMany` returns plain JavaScript objects with keys that are the raw column aliases from the SQL projection. Use `getRawMany` when the query includes aggregate expressions or projections across multiple entities that do not map to a single entity class.

7. When using a subquery inside a QueryBuilder WHERE clause, what additional step is required?

Expected answer: You must call `setParameters(subQuery.getParameters())` on the outer query builder. The subquery's named parameters (e.g., `:pub`) are defined on the subquery instance. Without transferring them to the outer builder, the outer query references those parameter names but the driver has no bindings for them, resulting in a runtime error.

8. What does `getSql()` do, and how is it different from `printSql()` ?

Expected answer: `getSql()` returns the generated SQL string without executing the query — useful for inspection or logging before committing to a database call. `printSql()` is a chainable method that logs the SQL to the console at the moment `getMany()` / `execute()` is called, without interrupting the query flow.

Practical Assignment — Analytics Query Service

Objective: Build a service class that exposes four analytical queries for the blog domain, all implemented with QueryBuilder.

Estimated time: 45–60 minutes

Setup

Use the project from Section 11-03 (or a fresh project with the `User` and `Post` entities from §2 of this document). Seed the database with at least three users and six posts (mix of published and unpublished, varying view counts).


```
blog-api/
├─ src/
│   ├─ entity/
│   │   ├─ User.ts
│   │   └─ Post.ts
│   ├─ service/
│   │   └─ AnalyticsService.ts ← create this
│   ├─ data-source.ts
│   └─ seed.ts ← create this for test data
```

Tasks

1. Create `src/seed.ts` that inserts at least 3 users and 6 posts (some published, some not, varying `viewCount` values) using the repository API. Run it once with `npx ts-node src/seed.ts`.
2. Create `src/service/AnalyticsService.ts` with the following methods, all using QueryBuilder:
 - `getTopPosts(limit: number): Promise<Post[]>` — returns the top N published posts by `viewCount` descending, with the author's `name` loaded via `leftJoinAndSelect`.
 - `getAuthorStats(): Promise<{ userName: string; postCount: string; totalViews: string }[]>` — for each user, returns their name, total number of posts (published and unpublished), and total view count. Use GROUP BY and `getRawMany`.
 - `searchPosts(opts: { search?: string; authorId?: number; published?: boolean }): Promise<Post[]>` — dynamic filter: apply each provided option as an `andWhere` condition, ignore undefined options.
 - `getAuthorsWithPublishedPosts(): Promise<User[]>` — use a subquery (IN pattern from §10) to return only users who have at least one published post.
3. Wire a test runner in `src/index.ts`: initialise the `DataSource`, instantiate `AnalyticsService`, call each method, and log the results.

Expected Output (example values — yours will differ based on seed data)

Output:

```
— Top 3 posts —
TypeORM Relations (450 views) by Carol Smith
Node.js Event Loop (312 views) by Dave
```

Express Middleware (200 views) by Carol Smith

— Author stats —

Carol Smith – 3 posts, 762 total views

Dave – 2 posts, 410 total views

Eve – 1 post, 0 total views

— Search: published + "node" —

Node.js Event Loop

Node Streams

— Authors with published posts —

Carol Smith

Dave

Bonus Challenge: Add a `getPostsPage(page: number, pageSize: number)` method that returns a paginated list of published posts. The tricky part: also return a `totalCount` without issuing a second query. Use `getManyAndCount()`, which returns a tuple `[Post[], number]` in a single round-trip. Log both the page items and the total count to verify pagination is correct across multiple pages.